



وثيقة تصميم النظام

منصة المتجر الإلكتروني

E-Commerce Platform — System Design & Architecture Document

Modular Monolith — NestJS · MySQL · Prisma · Docker · Stripe

1. نظرة عامة ومتطلبات العمل (System Overview)

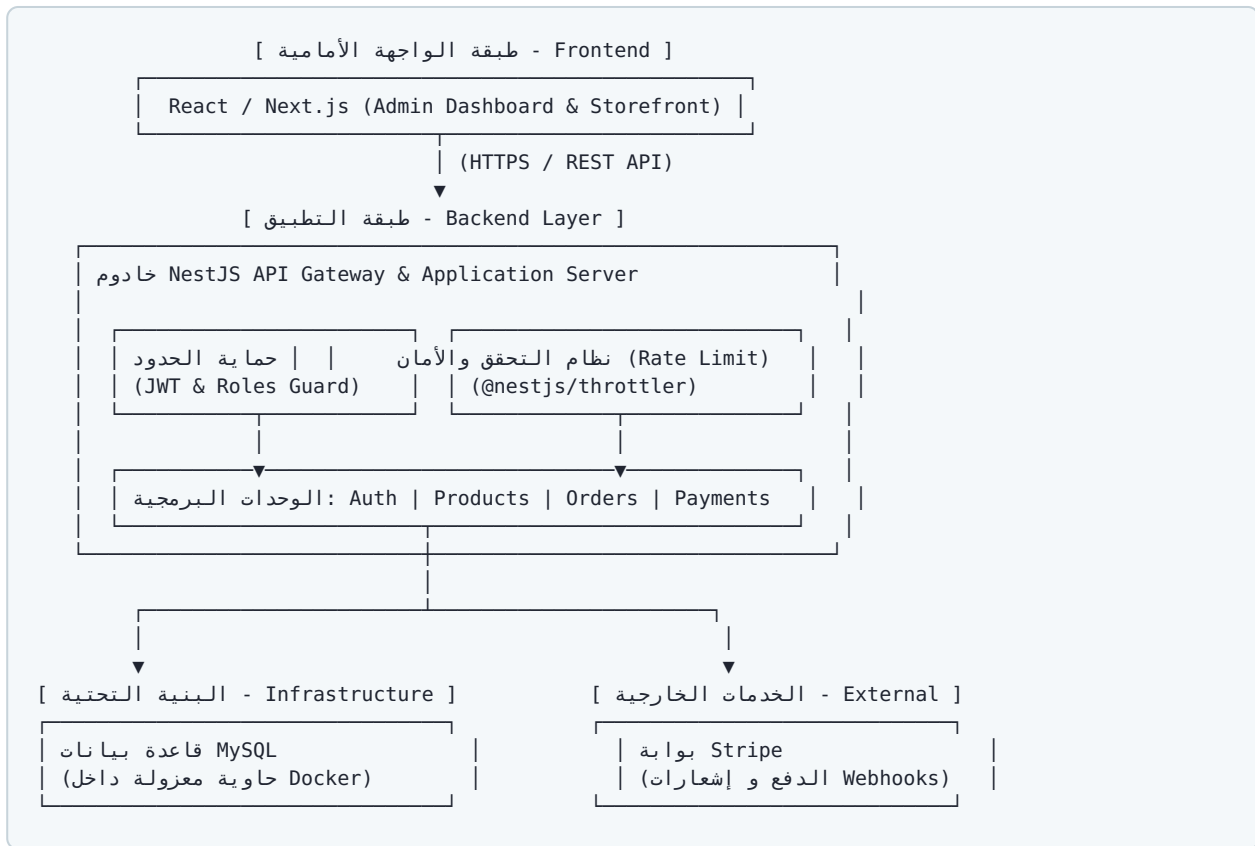
النظام عبارة عن منصة متجر إلكتروني متكاملة اعتمدت معمارية Modular Monolith باستخدام NestJS و MySQL، وتغطي الآتي:

- جهة الزبون (Customer Facing): تصفح المنتجات، السلة، إنشاء الطلبات، والدفع عبر Stripe.
- لوحة التحكم (Admin Dashboard): إدارة المنتجات، تتبع المخزون، وتحديث حالات الطلبات.

الأهداف التقنية الأساسية: الأمان (Authentication & RBAC)، الاعتمادية في الدفع (Idempotent Webhooks)، وإدارة المخزون أثناء الضغط العالي (Race Conditions).

2. الهيكلية العامة للنظام (High-Level Architecture)

تعتمد المعمارية على فصل الطبقات (Layered Architecture) لضمان سهولة الصيانة والتوسع مستقبلاً.



3. القرارات التقنية والمفاضلات (Architectural Trade-offs)

المكوّن	الخيار المُعتمد	السبب والمنطق (Justification)
المعمارية العامة	Modular Monolith	تفادي تعقيدات Microservices المعقدة في البداية، مع فصل الكود داخل Modules ليسهل تحويله لـ Microservices مستقبلاً.

المكوّن	الخيار المُعتمد	السبب والمنطق (Justification)
قاعدة البيانات	MySQL (عبر Prisma ORM)	توفر خصائص ACID الكاملة الضرورية لسلامة المعاملات المالية والمخزون.
نظام المصادقة	JWT + Refresh Tokens	Token للوصول قصير المدى (15 دقيقة) + Refresh Token محمي داخل HttpOnly Cookie.
الحاويات البرمجية	Docker & Docker Compose	عزل بيئة التطوير المحلية لتطابق بيئة الإنتاج الفعلية (Production Parity).

4. ضوابط الأمان والاعتمادية (Security & Reliability)

أ. ضوابط الأمان (Security Controls)

المصادقة والترخيص:

- AuthGuard : التحقق من صحة الـ Access Token.
 - RolesGuard : استخدام @Roles(Role.ADMIN) لتحديد صلاحية الوصول للوحة التحكم فقط.
- تنظيف البيانات (Sanitization): استخدام ValidationPipe مع class-validator لمنع هجمات SQL Injection و XSS.
- الحد من الطلبات (Rate Limiting): حماية مسارات تسجيل الدخول لمنع هجمات التخمين (Brute-force).

ب. التعامل مع تنافس الطلبات على المخزون (Race Conditions)

عند وجود طلبين شراء لمنتج في الوقت نفسه وتبقى منه قطعة واحدة فقط:

المشكلة: قد يقرأ الطلبان `stock = 1` ويقومان بالشراء معاً، فتصبح الكمية 1- (Over-selling).

الحل: استخدام Database Transactions مع التحديث المباشر للكمية (Atomic Updates):

```
UPDATE Product
SET stock = stock - :quantity
WHERE id = :productId AND stock >= :quantity;
```

ج. أمان إشعارات (Webhooks & Idempotency)

- التحقق من التوقيع (Signature Verification): التأكد من التوقيع الرقمي لمنع الطلبات المزيفة.
- التعامل مع التكرار (Idempotency): التحقق من حالة الطلب في قاعدة البيانات قبل تعديله لضمان عدم تنفيذ نفس الإشعار أكثر من مرة إذا أعاد Stripe إرساله.

5. مخطط قاعدة البيانات (Database Schema)

العلاقات بين الكيانات (ERD Relationships)

- المستخدم (User) ——— (عدة) طلبات (Order): علاقة 1 لـ N.
- الطلب (Order) ——— (عدة) عناصر طلب (OrderItem): علاقة 1 لـ N.
- المنتج (Product) ——— (عدة) عناصر طلب (OrderItem): علاقة 1 لـ N.

```
datasource db {
  provider = "mysql"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

// أدوار المستخدمين
enum Role {
  ADMIN
  CUSTOMER
}

// حالات الطلب
enum OrderStatus {
  PENDING
  PAID
  CANCELLED
  FAILED
}

// جدول المستخدمين
model User {
  id          String   @id @default(uuid())
  email       String   @unique
  password    String
  role        Role     @default(CUSTOMER)
  refreshToken String?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  orders      Order[]
}

// جدول المنتجات والمخزون
model Product {
  id          String   @id @default(uuid())
  title       String
  description  String?  @db.Text
  price       Decimal  @db.Decimal(10, 2)
  stock       Int       @default(0)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  orderItems  OrderItem[]
}
```

```
// جدول الطلبات الرئيسي
model Order {
  id          String      @id @default(uuid())
  userId      String
  user        User        @relation(fields: [userId], references: [id])
  totalAmount Decimal     @db.Decimal(10, 2)
  status      OrderStatus @default(PENDING)
  stripePaymentId String?  @unique
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt
  items       OrderItem[]
}

// جدول تفاصيل عناصر الطلب
model OrderItem {
  id          String      @id @default(uuid())
  orderId     String
  order       Order        @relation(fields: [orderId], references: [id], onDelete: Cascade)
  productId   String
  product     Product      @relation(fields: [productId], references: [id])
  quantity    Int
  price       Decimal     @db.Decimal(10, 2)
}
```

6. بيئة التشغيل والحاويات (Docker Setup)

ملف `docker-compose.yml` لتشغيل الخادوم وقاعدة البيانات محلياً داخل حاويات معزولة:

```
version: '3.8'

services:
  mysql_db:
    image: mysql:8.0
    container_name: ecommerce_mysql
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpassword
      MYSQL_DATABASE: ecommerce_db
      MYSQL_USER: nestuser
      MYSQL_PASSWORD: nestpassword
    ports:
      - '3306:3306'
    volumes:
      - mysql_data:/var/lib/mysql

  backend:
    build: .
    container_name: ecommerce_backend
    restart: always
    ports:
      - '3000:3000'
    environment:
      DATABASE_URL: "mysql://nestuser:nestpassword@mysql_db:3306/ecommerce_db"
      JWT_SECRET: "super-secret-key"
      STRIPE_SECRET_KEY: "sk_test_..."
    depends_on:
```

```
- mysql_db  
  
volumes:  
  mysql_data:
```